



**Universidad Nacional
Autónoma de México**



Facultad de Ingeniería

**Laboratorio de Arquitectura y Organización
de Computadoras**

Grupo: 07 - Semestre: 2027-1

Reporte Practica 2 Pt.1

Diseño de máquinas de estado

Fecha de Entrega

04 de Septiembre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Familiarizar al alumno en el conocimiento de los algoritmos de las máquinas de estados utilizando Quartus y el lenguaje VHDL.

2. Desarrollo

A partir de la carta ASM que se proporciona en la práctica (Figura [1]) se solicita obtener su circuito secuencial utilizando flip-flops tipo D. Además se debe de implementar el circuito en el ambiente de desarrollo Quartus.

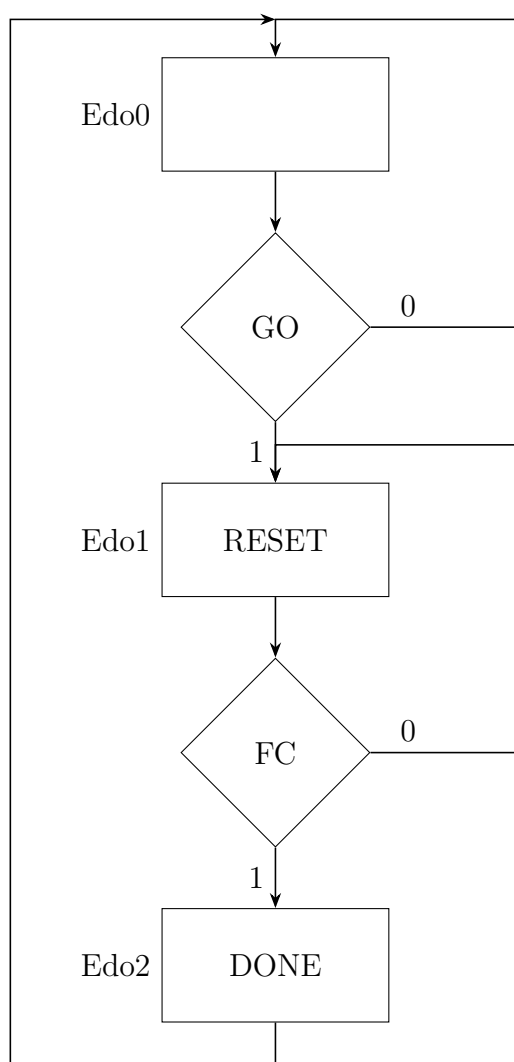


Figura 1: Carta ASM Práctica 2

Para obtener el circuito secuencial a partir de la carta ASM, se debe de realizar la tabla de transición de estados y la tabla de salida:

	Estado Presente		Entradas		Estado Siguiende		Salidas	
	Q_1	Q_0	GO	FC	Q_1^+	Q_0^+	$RESET$	$DONE$
Edo0	0	0	0	*	0	0	0	0
	0	0	1	*	0	1	0	0
Edo1	0	1	*	0	0	1	1	0
	0	1	*	1	1	0	1	0
Edo2	1	0	*	*	0	0	0	1

Tabla 1: Tabla de Transición de Estados

Obteniendo las ecuaciones por medio de mapas de Karnaugh, se obtiene lo siguiente:

Reducción por medio de Mapas de Karnaugh: Q_0^+

		$Q_1 Q_0$			
		00	01	11	10
$GO\ FC$	00	0	0	1	1
	01	1	0	0	1
	11	X	X	X	X
	10	0	0	0	0

$$Q_0^+ = Q_0 \overline{FC} + \overline{Q_1} Q_0 GO$$

Reducción por medio de Mapas de Karnaugh: Q_1^+

		Q_1Q_0			
		00	01	11	10
<i>GO FC</i>	00	0	0	1	1
	01	1	0	0	1
	11	X	X	X	X
	10	0	0	0	0

$$Q_1^+ = Q_0 + FC$$

Reducción por medio de Mapas de Karnaugh: *RESET*

		Q_1Q_0			
		00	01	11	10
<i>GO FC</i>	00	0	0	0	0
	01	1	1	1	1
	11	X	X	X	X
	10	0	0	0	0

$$RESET = Q_0$$

Reducción por medio de Mapas de Karnaugh: *DONE*

		Q_1Q_0			
		00	01	11	10
$GO FC$	00	0	0	0	0
	01	0	0	0	0
	11	X	X	X	X
	10	1	1	1	1

$DONE = Q_1$

3. Simulaciones

La verificación funcional se realizó en el editor de formas de onda de Quartus. Se definieron como entradas el reloj `clock` y la señal de inicio `go`; como salidas se observaron `A`, `B`, `C`, `D` y `done`. El reloj se aplicó de forma periódica y `go` se mantuvo en alto para solicitar la operación del circuito. Esta configuración permite observar de manera continua la transición de la máquina de estados, el conteo y la indicación de terminación.

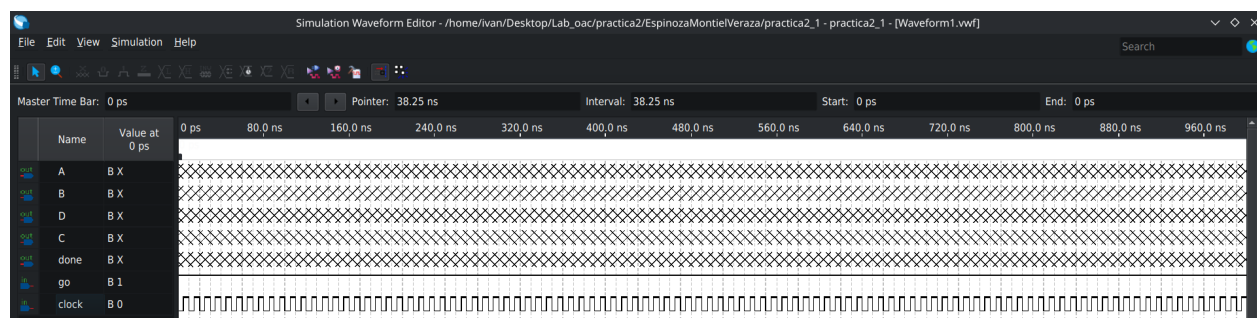


Figura 2: Condiciones iniciales de la simulación. Antes de propagarse la inicialización, las salidas se muestran como valores indefinidos.

En la Figura 2 se aprecia el estado previo a la evaluación del diseño: las salidas aparecen como X porque todavía no se ha establecido un estado lógico válido para los elementos secuenciales. Este comportamiento es normal en una simulación temporal cuando no se ha aplicado aún un flanco de reloj que propague las condiciones de inicio. A partir de los flancos activos del reloj, la máquina de estados abandona esta condición y las señales toman valores binarios definidos.

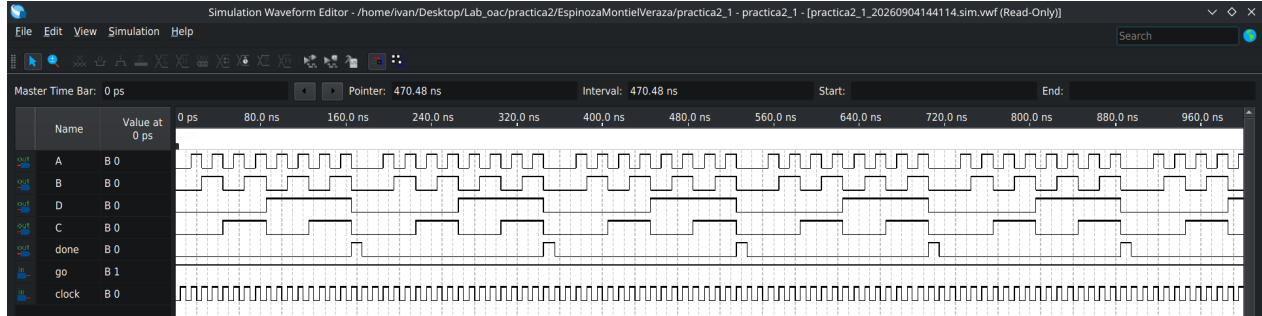


Figura 3: Simulación funcional completa con $go = 1$.

La Figura 3 confirma el comportamiento esperado del contador de cuatro bits. La salida A cambia de estado con mayor frecuencia, B lo hace cada dos cambios de A, C cada cuatro y D cada ocho; por tanto, en conjunto forman la secuencia binaria ascendente de 0000 a 1111. Esta relación de frecuencias muestra que cada salida representa un bit de distinta ponderación del conteo. Al alcanzarse el conteo terminal, la lógica de detección genera un pulso en done; dicho pulso es breve y aparece sincronizado con el evento de fin de cuenta.

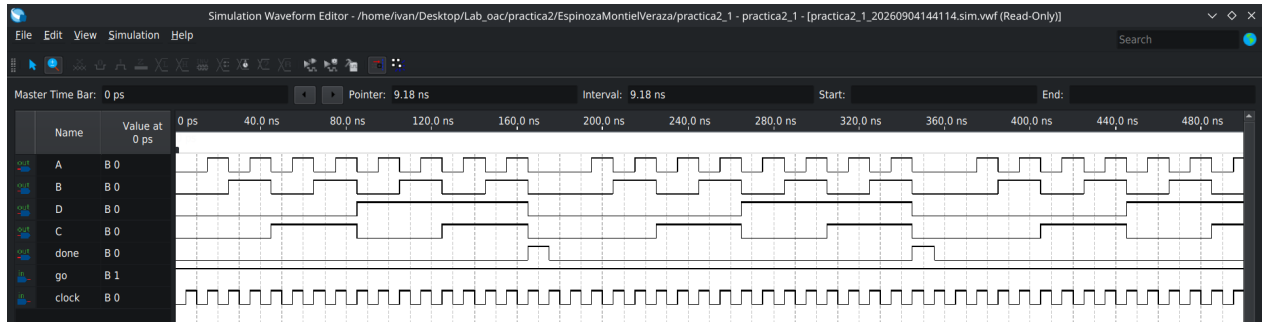


Figura 4: Vista ampliada de la secuencia de conteo y del pulso done.

En la vista ampliada de la Figura 4 se distinguen con claridad los cambios simultáneos de los bits al pasar entre valores consecutivos. La señal done sólo se activa cuando la combinación de las salidas cumple la condición de cuenta final; inmediatamente después, el circuito vuelve a una condición de conteo válida y el ciclo se repite mientras go permanezca activo. Con ello se comprueba que la salida de terminación no permanece habilitada durante todo el ciclo, sino que funciona como una notificación de un periodo de reloj.

4. Resultados

El diseño se implementó como un circuito jerárquico compuesto por un divisor de frecuencia, una máquina de estados construida con flip-flops tipo D, lógica combinacional y un contador

de cuatro bits. Para la verificación física se cargó el diseño en la tarjeta FPGA mediante *Tools* → *Programmer*. El divisor de frecuencia es indispensable para reducir la frecuencia del reloj de la tarjeta a una velocidad apreciable en las salidas del contador.

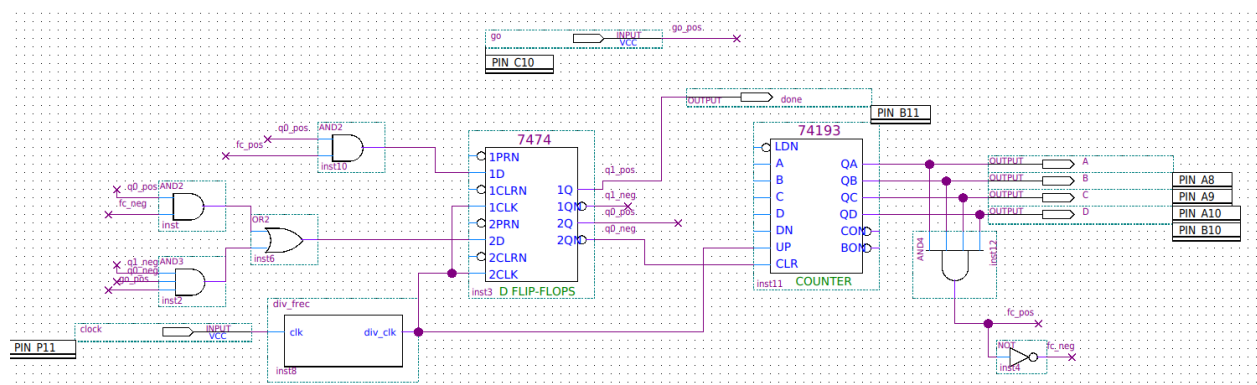


Figura 5: Diagrama de bloques completo implementado en Quartus.

En la Figura 5 se muestra la integración final. La entrada `clock` llega al bloque `div_freq`; su salida temporizada alimenta tanto los flip-flops 7474 como el contador 74193. La entrada `go` inicia la secuencia de la máquina de estados. Las señales de estado controlan las entradas del contador, mientras que las salidas A, B, C y D se conectan a los pines de salida de la FPGA. La compuerta AND detecta la condición de cuenta final y genera la realimentación `fc`; su complemento también se utiliza en la lógica de transición. La señal `done` se obtiene del estado correspondiente de la máquina.

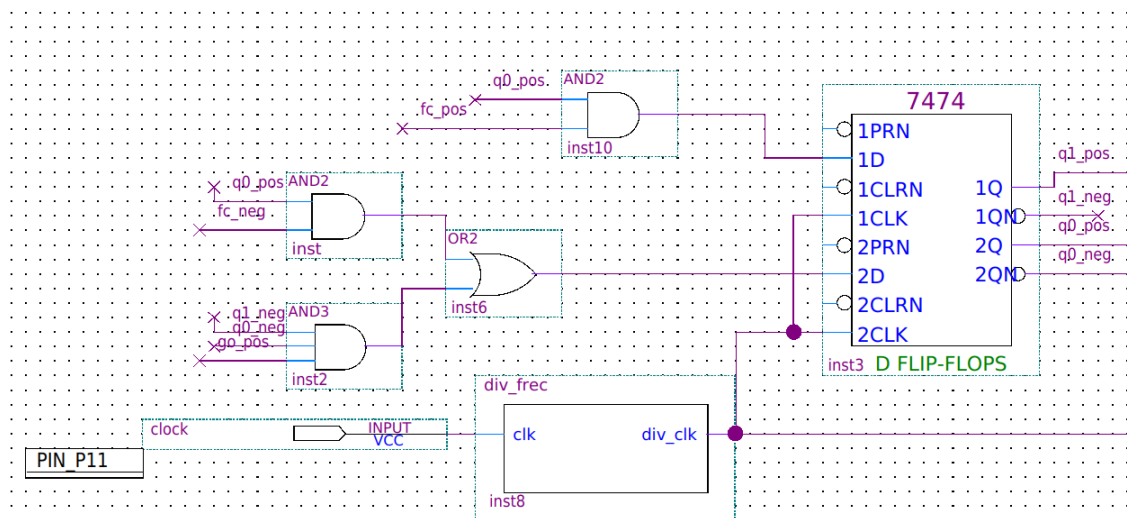


Figura 6: Detalle de la sección de control: divisor de frecuencia, lógica combinacional y flip-flops D 7474.

La Figura 6 corresponde a la sección que implementa la carta ASM. Los dos flip-flops D almacenan las variables de estado Q_1 y Q_0 . Las compuertas AND y OR materializan las ecuaciones simplificadas mediante los mapas de Karnaugh, y sus salidas determinan el siguiente estado en cada flanco de reloj. De esta forma, el circuito pasa del estado de espera al estado de reinicio y posteriormente al estado de término según los valores de *go* y *fc*.

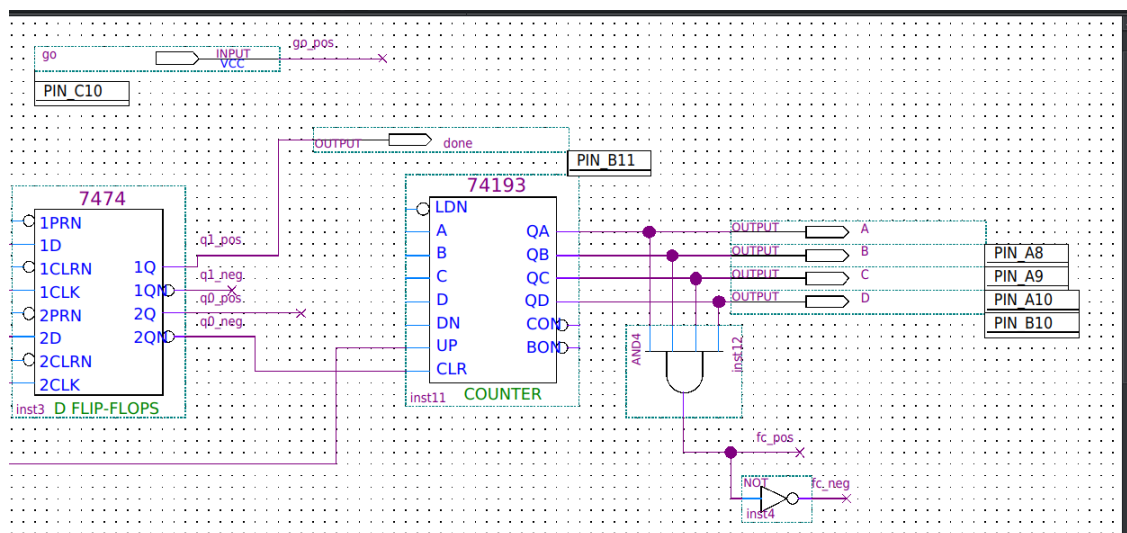
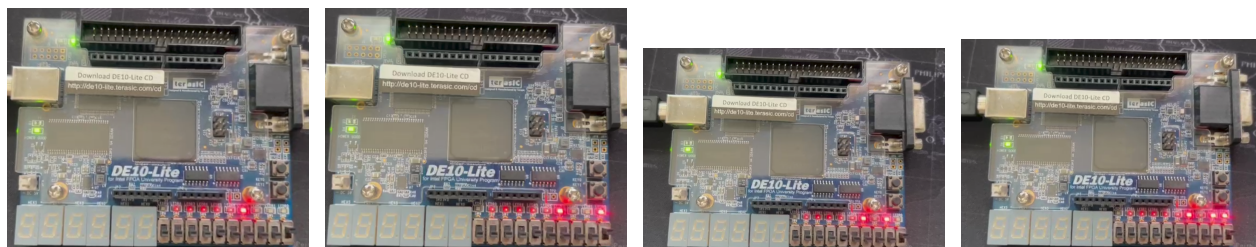


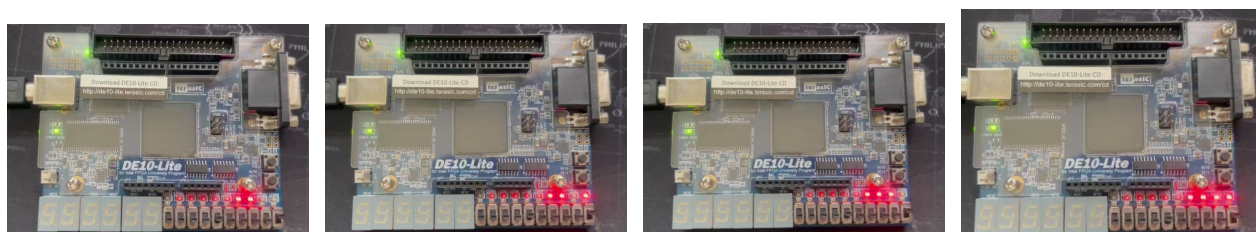
Figura 7: Detalle de la sección de conteo y detección de cuenta final.

En la Figura 7, el contador 74193 entrega los cuatro bits de salida. La compuerta AND de cuatro entradas reconoce el valor máximo del contador al recibir simultáneamente A, B, C y D en alto; así se forma la señal *fc*. El inversor produce *fc_neg*, necesaria para la lógica de control. Esta conexión permite cerrar el ciclo: se cuenta mientras no se haya alcanzado la condición final y, al detectarla, la máquina de estados genera *done* y prepara el reinicio del proceso.

Contador de 4 bits cuando $\text{reset} = 0$



(a) Contador de 4 bits (1000) (b) Contador de 4 bits (1001) (c) Contador de 4 bits (1010) (d) Contador de 4 bits (1011)



(e) Contador de 4 bits (1100) (f) Contador de 4 bits (1101) (g) Contador de 4 bits (1110) (h) Contador de 4 bits (1111)

Figura 8: Algunos valores del Contador de 4 bits cuando $\text{reset} = 0$.

Contador de 4 bits cuando $\text{reset} = 1$

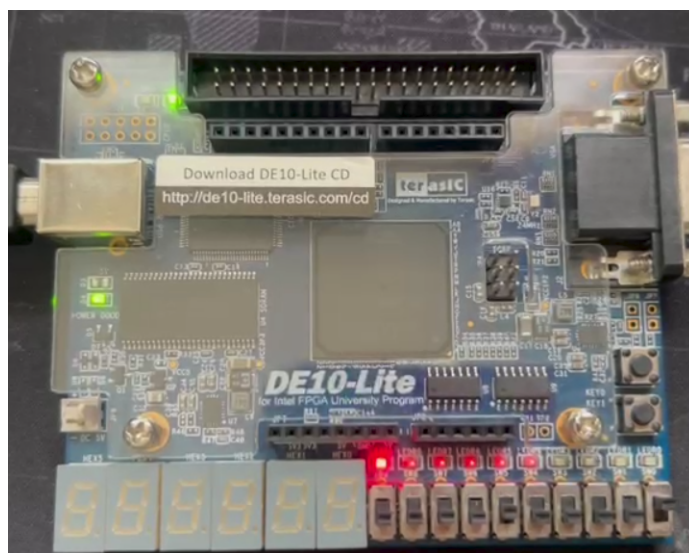


Figura 9: Contador de 4 bits cuando $\text{reset} = 1$. (0000)

5. Conclusión

Percival Ulises Espinoza Matamoros: La práctica permitió relacionar la carta ASM con su implementación física. A partir de las tablas de transición y de salida se obtuvieron ecuaciones que pueden llevarse directamente a compuertas y flip-flops. Esto mostró que una máquina de estados no es sólo una descripción abstracta: sus estados y transiciones se materializan en señales que pueden observarse y verificarse.

Oscar Iván Montiel Juárez: La simulación fue fundamental para validar el diseño antes de programar la FPGA. Las formas de onda confirmaron la secuencia binaria del contador, la diferencia de frecuencia entre sus bits y la generación puntual de **done** al llegar a la cuenta terminal. Además, observar los valores indefinidos al inicio permitió reconocer la importancia de inicializar y sincronizar adecuadamente los elementos secuenciales.

Amy Valentina Veraza García: La integración del divisor de frecuencia hizo posible trasladar el diseño de simulación a la tarjeta FPGA de forma observable. El resultado final combina control secuencial, lógica combinatorial y conteo en un sistema funcional. La práctica reforzó el uso de Quartus y VHDL como herramientas para diseñar, simular e implementar circuitos digitales de manera ordenada.